

Objetivos da aula:

- Compreender o conceito de padrões de projeto e sua importância no desenvolvimento de software.
- Conhecer a classificação dos padrões GoF, com foco nos padrões criacionais.
- Entender o funcionamento dos padrões Factory, Singleton, Builder, Abstract Factory e Prototype.
- Analisar como esses padrões contribuem para melhorar a organização, flexibilidade e manutenção do código.
- Aplicar padrões criacionais em exemplos práticos utilizando TypeScript.

Índice:

- i. Introdução aos Padrões de Projeto
- ii. Factory Pattern
- iii. Singleton Pattern
- iv. Builder Pattern
- v. Abstract Factory Pattern
- vi. Prototype Pattern

i. Introdução aos Padrões de Projeto

1.1. Introdução

No desenvolvimento de software, a forma como organizamos o código influencia diretamente a **qualidade, manutenção e evolução de um sistema**.

No desenvolvimento de software, muitos problemas de design aparecem repetidamente em diferentes projetos.

Esses problemas podem envolver, por exemplo:

- organização de classes
- comunicação entre objetos
- criação de objetos complexos
- reutilização de código

Ao longo do tempo, desenvolvedores perceberam que determinadas soluções funcionavam bem para resolver esses problemas recorrentes. Essas soluções passaram a ser documentadas e compartilhadas na forma de **padrões de projeto (Design Patterns)**.

Um **padrão de projeto** pode ser definido como uma **solução geral e reutilizável para um problema comum no design de software**.

É importante destacar que um padrão de projeto **não é um código pronto**, mas sim um **modelo de solução** que pode ser adaptado para diferentes situações.

O uso de padrões de projeto contribui para a construção de sistemas que são:

- mais organizados

- mais flexíveis
- mais fáceis de manter
- mais fáceis de evoluir

1.2. Origem dos Padrões de Projeto

Os padrões de projeto tornaram-se amplamente conhecidos após a publicação, em 1994, do livro:

“Design Patterns: Elements of Reusable Object-Oriented Software”

Os autores do livro são:

- Erich Gamma
- Richard Helm
- Ralph Johnson
- John Vlissides

Esses autores ficaram conhecidos como Gang of Four (GoF).

O livro descreve 23 padrões de projeto voltados para o desenvolvimento de software orientado a objetos, organizados em diferentes categorias.

Desde então, esses padrões passaram a ser amplamente utilizados na engenharia de software e influenciaram diversas linguagens, frameworks e arquiteturas modernas.

1.3. Classificação dos Padrões GoF

Os padrões apresentados pela Gang of Four são classificados em **três categorias principais**, de acordo com o tipo de problema que procuram resolver.

1.3.1. Padrões Criacionais

Os padrões criacionais tratam da **forma como os objetos são criados em um sistema**.

Eles procuram tornar a criação de objetos mais flexível e desacoplada do restante da aplicação.

Exemplos:

1. Factory Method: delegar a criação de objetos
2. Abstract Factory: criar famílias de objetos
3. Builder: construir objetos complexos passo a passo
4. Prototype: criar objetos por clonagem
5. Singleton: garantir uma única instância

1.3.2. Padrões Estruturais

Os padrões estruturais tratam da **organização e composição de classes e objetos**.

Eles ajudam a estruturar sistemas complexos combinando diferentes componentes.

Exemplos:

6. Adapter: adaptar interfaces incompatíveis
7. Bridge: separar abstração da implementação
8. Composite: tratar objetos e grupos de forma uniforme
9. Decorator: adicionar comportamento dinamicamente
10. Facade: simplificar acesso a um subsistema
11. Flyweight: compartilhar objetos para economizar memória
12. Proxy: controlar acesso a um objeto

1.3.3. Padrões Comportamentais

Os padrões comportamentais tratam da **forma como objetos interagem e se comunicam entre si**.

Esses padrões ajudam a organizar responsabilidades e fluxos de comunicação dentro do sistema.

Exemplos:

13. Chain of Responsibility: passar requisição por uma cadeia
14. Command: encapsular comandos em objetos
15. Interpreter: interpretar linguagem ou expressões
16. Iterator: percorrer elementos de uma coleção
17. Mediator: centralizar comunicação entre objetos
18. Memento: salvar e restaurar estado
19. Observer: notificar objetos sobre mudanças
20. State: alterar comportamento conforme estado
21. Strategy: trocar algoritmos dinamicamente
22. Template Method: definir esqueleto de algoritmo
23. Visitor: adicionar operações a estruturas

1.4. Importância dos Padrões de Projeto

O uso de padrões de projeto traz diversas vantagens para o desenvolvimento de software.

Entre os principais benefícios estão:

- soluções baseadas em práticas consolidadas
- maior padronização do código
- melhoria na comunicação entre desenvolvedores
- maior reutilização de soluções
- sistemas mais flexíveis e extensíveis

Além disso, muitos frameworks modernos utilizam internamente diversos padrões de projeto.

Compreender padrões de projeto ajuda o desenvolvedor a entender melhor **como frameworks e bibliotecas são estruturados**.

ii. Factory Pattern

2.1. Introdução

Em muitos sistemas, objetos são criados diretamente utilizando a instrução `new`. Em aplicações pequenas isso não costuma gerar problemas, porém em sistemas maiores essa prática pode dificultar a manutenção do código.

```
class NotificacaoService {
    enviar(tipo: string) {
        if (tipo === "email") {
            console.log("Enviando email...");
        } else if (tipo === "sms") {
            console.log("Enviando SMS...");
        }
    }
}
```

Nesse exemplo, a classe `NotificacaoService` precisa decidir **qual tipo de notificação utilizar**.

À medida que novos tipos de notificação são adicionados, o código tende a crescer e se tornar mais difícil de manter. Esse tipo de problema ocorre porque **a lógica de criação dos objetos está misturada com a lógica de uso desses objetos**.

Uma solução mais adequada consiste em **separar a responsabilidade de criação dos objetos em uma estrutura específica**.

Esse é o objetivo do Factory Pattern.

2.2. O que é o Factory Pattern

O Factory Pattern é um padrão criacional que tem como objetivo **centralizar e abstrair a criação de objetos**.

Em vez de criar objetos diretamente utilizando `new`, a criação é delegada a uma **classe ou método responsável por instanciar os objetos apropriados**.

Esse padrão traz algumas vantagens importantes:

- reduz dependências entre classes
- facilita a manutenção do código
- torna o sistema mais extensível
- centraliza a lógica de criação de objetos

2.3. Estrutura do Factory Pattern

De forma geral, o Factory Pattern envolve os seguintes elementos:

Elemento	Função
Interface ou classe base	define o comportamento comum

Elemento	Função
Classes concretas	implementam o comportamento
Factory	responsável por criar os objetos

2.4. Exemplo Simples em TypeScript

Considere um sistema de notificações que pode enviar mensagens por email ou SMS.

Interface comum

```
interface Notificacao {  
    enviar(mensagem: string): void;  
}
```

Implementações

```
class EmailNotificacao implements Notificacao {  
    enviar(mensagem: string) {  
        console.log("Email enviado:", mensagem);  
    }  
}
```

```
class SmsNotificacao implements Notificacao {  
    enviar(mensagem: string) {  
        console.log("SMS enviado:", mensagem);  
    }  
}
```

Factory

```
class NotificacaoFactory {  
    static criar(tipo: string): Notificacao {  
        if (tipo === "email") {  
            return new EmailNotificacao();  
        } else if (tipo === "sms") {  
            return new SmsNotificacao();  
        }  
  
        throw new Error("Tipo de notificação inválido");  
    }  
}
```

Uso

```
const notificacao = NotificacaoFactory.criar("email");  
notificacao.enviar("Bem-vindo ao sistema");
```

Nesse caso, o código que utiliza o objeto **não precisa saber qual classe está sendo instanciada**.

2.5. Benefícios do Factory Pattern

O uso do Factory Pattern traz diversas vantagens:

- centralização da criação de objetos
- redução de dependências diretas entre classes
- facilidade para adicionar novos tipos de objetos
- maior organização do código

Esse padrão é muito utilizado em aplicações desenvolvidas com:

- Node.js + TypeScript
- frameworks backend
- bibliotecas de serviços
- sistemas que possuem múltiplas implementações para um mesmo comportamento.

iii. Singleton Pattern

3.1. Introdução

Em alguns sistemas, é importante garantir que **apenas uma instância de determinada classe exista durante toda a execução da aplicação**.

Isso ocorre, por exemplo, em situações como:

- conexão com banco de dados
- sistema de logging
- configuração global da aplicação
- gerenciadores de cache

Se múltiplas instâncias dessas estruturas forem criadas, podem ocorrer problemas como:

- desperdício de recursos
- inconsistência de dados
- comportamentos inesperados no sistema

Para resolver esse tipo de situação, pode-se utilizar o Singleton Pattern.

3.2. O que é o Singleton Pattern

O Singleton Pattern é um padrão criacional que garante que **uma classe possua apenas uma única instância**, fornecendo um ponto global de acesso a essa instância.

Esse padrão possui duas características principais:

1. A classe controla a criação da própria instância
2. A aplicação utiliza sempre a mesma instância

Dessa forma, o sistema evita a criação de múltiplos objetos para algo que deve existir apenas uma vez.

3.3. Estrutura do Singleton

De forma geral, o Singleton possui os seguintes elementos:

Elemento	Função
Construtor privado	impede criação direta com new
Instância estática	armazena a única instância da classe
Método de acesso	retorna a instância única

3.4. Exemplo Simples em TypeScript

Considere uma classe responsável por registrar mensagens de log.

```
class Logger {  
    private static instancia: Logger;  
  
    private constructor() {}  
  
    static getInstance(): Logger {  
        if (!Logger.instancia) {  
            Logger.instancia = new Logger();  
        }  
  
        return Logger.instancia;  
    }  
  
    log(mensagem: string) {  
        console.log("LOG:", mensagem);  
    }  
}
```

Uso

```
const logger1 = Logger.getInstance();  
const logger2 = Logger.getInstance();  
logger1.log("Aplicação iniciada");  
console.log(logger1 === logger2); // true
```

Nesse exemplo, independentemente de quantas vezes `getInstance()` seja chamado, **a mesma instância será utilizada**.

3.5. Cuidados no Uso do Singleton

Embora seja um padrão útil, o Singleton deve ser utilizado com cuidado.

O uso excessivo pode gerar alguns problemas, como:

- dependências globais no sistema
- dificuldade em testes automatizados
- aumento do acoplamento entre partes da aplicação

Por isso, é importante utilizar esse padrão apenas quando **realmente for necessário garantir uma única instância**.

iv. Builder Pattern

4.1. Introdução

Em muitas aplicações, é comum trabalhar com objetos que possuem **muitos atributos ou diferentes configurações possíveis**.

Quando esses objetos são criados diretamente pelo construtor da classe, o código pode se tornar difícil de entender e manter.

Considere o exemplo a seguir em TypeScript:

```
class Usuario {  
  constructor(  
    public nome: string,  
    public email: string,  
    public idade: number,  
    public endereco: string,  
    public telefone: string,  
  ) {}  
}  
  
const usuario = new Usuario("Ana", "ana@email.com", 25, "Rua A", "99999-9999");
```

À medida que o número de parâmetros aumenta, o código se torna:

- difícil de ler
- propenso a erros
- difícil de manter

Além disso, muitas vezes alguns desses atributos podem ser **opcionais**, o que torna a criação do objeto ainda mais complexa.

Para resolver esse problema, pode-se utilizar o Builder Pattern.

4.2. O que é o Builder Pattern

O Builder Pattern é um padrão criacional utilizado para **construir objetos complexos passo a passo**.

Em vez de criar o objeto diretamente utilizando um construtor com muitos parâmetros, o Builder permite **definir cada propriedade de forma gradual**.

Esse padrão separa:

- o processo de construção do objeto
- a representação final do objeto

Isso torna o código mais legível e mais flexível.

4.3. Estrutura do Builder Pattern

De forma geral, o Builder envolve os seguintes elementos:

Elemento	Função
Objeto principal	classe que será construída
Builder	responsável por construir o objeto
Métodos de configuração	definem as propriedades do objeto
Método build()	retorna o objeto final

4.4. Exemplo Simples em TypeScript

Primeiro definimos a classe principal:

```
class Usuario {  
  nome?: string;  
  email?: string;  
  idade?: number;  
  endereco?: string;  
}
```

Agora criamos o Builder responsável por construir esse objeto.

```
class UsuarioBuilder {  
  private usuario: Usuario = new Usuario();  
  
  setNome(nome: string): UsuarioBuilder {  
    this.usuario.nome = nome;  
    return this;  
  }  
  
  setEmail(email: string): UsuarioBuilder {  
    this.usuario.email = email;  
    return this;  
  }  
  
  setIdade(idade: number): UsuarioBuilder {  
    this.usuario.idade = idade;  
    return this;  
  }  
  
  setEndereco(endereco: string): UsuarioBuilder {  
    this.usuario.endereco = endereco;  
    return this;  
  }  
  
  build(): Usuario {  
    return this.usuario;  
  }  
}
```

Uso

```
const usuario = new UsuarioBuilder()
    .setNome("Ana")
    .setEmail("ana@email.com")
    .setIdade(25)
    .build();
```

Esse estilo de construção é chamado de **fluent interface**, pois os métodos podem ser encadeados.

4.5. Benefícios do Builder Pattern

O uso do Builder Pattern traz diversas vantagens:

- melhora a legibilidade do código
- evita construtores com muitos parâmetros
- permite criação gradual de objetos
- facilita a manutenção e extensão do sistema

Esse padrão aparece frequentemente em:

- bibliotecas de configuração
- criação de objetos complexos
- frameworks backend
- APIs de construção de objetos

v. Abstract Factory Pattern

5.1. Introdução

Em muitos sistemas, não precisamos apenas criar **um objeto específico**, mas sim **um conjunto de objetos relacionados que trabalham juntos**.

Considere, por exemplo, um sistema que precisa criar diferentes componentes de interface dependendo da plataforma utilizada:

- interface para web
- interface para aplicação mobile

Cada plataforma pode possuir seus próprios componentes, como:

Plataforma	Botão	Campo de texto
Web	WebButton	WebInput
Mobile	MobileButton	MobileInput

Nesse caso, não queremos apenas criar um objeto isolado, mas sim **uma família de objetos compatíveis entre si**.

Para resolver esse tipo de problema, pode-se utilizar o Abstract Factory Pattern.

5.2. O que é o Abstract Factory Pattern

O Abstract Factory Pattern é um padrão criacional que fornece uma **interface para criar famílias de objetos relacionados ou dependentes sem especificar suas classes concretas**.

Esse padrão permite que o sistema utilize objetos de uma mesma família sem precisar conhecer suas implementações específicas.

Em outras palavras, o Abstract Factory permite:

- criar **conjuntos de objetos relacionados**
- trocar facilmente **famílias inteiras de implementações**
- manter o código desacoplado das classes concretas

5.3. Estrutura do Abstract Factory

O padrão geralmente envolve os seguintes elementos:

Elemento	Função
Abstract Factory	interface que define métodos de criação
Concrete Factory	implementações específicas da fábrica
Abstract Products	interfaces dos objetos criados
Concrete Products	implementações concretas desses objetos

5.4. Exemplo em TypeScript

Considere um sistema que cria componentes de interface.

Interfaces dos produtos

```
interface Button {  
  render(): void;  
}
```

```
interface Input {  
  render(): void;  
}
```

Implementações para Web

```
class WebButton implements Button {  
  render() {  
    console.log("Renderizando botão web");  
  }  
}
```

```
class WebInput implements Input {  
  render() {  
    console.log("Renderizando input web");  
  }  
}
```

```
}  
}
```

Implementações para Mobile

```
class MobileButton implements Button {  
  render() {  
    console.log("Renderizando botão mobile");  
  }  
}  
  
class MobileInput implements Input {  
  render() {  
    console.log("Renderizando input mobile");  
  }  
}
```

Interface da fábrica

```
interface UIFactory {  
  createButton(): Button;  
  createInput(): Input;  
}
```

Fábrica para Web

```
class WebFactory implements UIFactory {  
  createButton(): Button {  
    return new WebButton();  
  }  
  
  createInput(): Input {  
    return new WebInput();  
  }  
}
```

Fábrica para Mobile

```
class MobileFactory implements UIFactory {  
  createButton(): Button {  
    return new MobileButton();  
  }  
  
  createInput(): Input {  
    return new MobileInput();  
  }  
}
```

Uso

```
function renderUI(factory: UIFactory) {  
  const button = factory.createButton();  
  const input = factory.createInput();  
}
```

```
        button.render();  
        input.render();  
    }  
  
    const factory = new WebFactory();  
    renderUI(factory);
```

Nesse exemplo, o código que utiliza os componentes **não precisa saber se está trabalhando com componentes Web ou Mobile**.

5.5. Benefícios do Abstract Factory

O uso desse padrão traz algumas vantagens importantes:

- separa a criação de objetos da lógica da aplicação
- garante que objetos compatíveis sejam utilizados juntos
- facilita a troca de implementações
- melhora a organização do código

Esse padrão é bastante utilizado em:

- frameworks de interface gráfica
- bibliotecas multiplataforma
- sistemas que suportam múltiplas implementações

5.6. Conclusão

O Abstract Factory Pattern é uma evolução da ideia apresentada no Factory Pattern.

Enquanto o Factory cria **um objeto específico**, o Abstract Factory permite criar **famílias inteiras de objetos relacionados**.

Esse padrão ajuda a tornar o sistema mais flexível e facilita a troca de implementações sem modificar o código principal da aplicação.

vi. Prototype Pattern

6.1. Introdução

Em alguns sistemas, a criação de objetos pode ser um processo **custoso ou complexo**, especialmente quando envolve:

- inicializações demoradas
- carregamento de dados
- múltiplas configurações

Nesses casos, pode ser mais eficiente **criar novos objetos a partir da cópia de um objeto já existente**, em vez de construir um novo objeto do zero.

Essa técnica é conhecida como **clonagem de objetos**.

O padrão de projeto que organiza essa abordagem é o Prototype Pattern.

6.2. O que é o Prototype Pattern

O Prototype Pattern é um padrão criacional que permite criar novos objetos **copiando (clonando) uma instância existente**, em vez de criar um objeto diretamente utilizando um construtor.

Esse padrão é útil quando:

- a criação de objetos é cara em termos de processamento
- muitas instâncias semelhantes precisam ser criadas
- queremos evitar dependência direta de classes concretas

Em vez de criar novos objetos com `new`, o sistema utiliza um **protótipo existente como base para gerar novas instâncias**.

6.3. Estrutura do Prototype

De forma geral, o padrão envolve os seguintes elementos:

Elemento	Função
Prototype	interface ou classe base que define o método de clonagem
Concrete Prototype	implementação concreta que pode ser clonada
Cliente	solicita novas cópias do objeto

O elemento central desse padrão é o **método de clonagem**, normalmente chamado de `clone()`.

6.4. Exemplo Simples em TypeScript

Considere uma classe que representa um usuário.

```
class Usuario {  
  constructor(  
    public nome: string,  
    public email: string,  
  ) {}  
  
  clone(): Usuario {  
    return new Usuario(this.nome, this.email);  
  }  
}
```

Uso

```
const usuario1 = new Usuario("Ana", "ana@email.com");
```

```
const usuario2 = usuario1.clone();  
usuario2.nome = "Carlos";  
console.log(usuario1);  
console.log(usuario2);
```

Nesse exemplo:

- usuario1 é o **protótipo**
- usuario2 é uma **cópia clonada**

A clonagem permite criar novos objetos com base em uma instância existente.

Exercícios

Veja os vídeos se tiver dúvidas:

Exercício 1: <https://youtu.be/XfxXHkjVmNY>

Exercício 2: <https://youtu.be/5SpNvPpZT5g>

Exercício 3: <https://youtu.be/eYOwcGODGgw>

Exercício 4: https://youtu.be/gB6_wH0ViR0

Exercício 5: <https://youtu.be/5f5fR2c3PTk>

Exercício 1 – Aplicando o Factory Pattern

Contexto

Considere um sistema de processamento de pagamentos em uma aplicação de e-commerce.

O sistema pode processar pagamentos utilizando diferentes métodos:

- Cartão de crédito
- Pix

A implementação inicial do sistema foi feita da seguinte forma:

```
class PagamentoService {  
  processarPagamento(tipo: string, valor: number) {  
    if (tipo === "cartao") {  
      console.log("Pagamento com cartão aprovado:", valor);  
    } else if (tipo === "pix") {  
      console.log("Pagamento via PIX aprovado:", valor);  
    }  
  }  
}
```

Parte A – Identificando o problema

Responda:

- a) Qual problema pode ocorrer se novos métodos de pagamento forem adicionados (por exemplo: boleto ou PayPal)?

b) Explique por que essa classe possui **baixa flexibilidade**.

Parte B – Criando uma interface

Crie uma interface chamada MetodoPagamento contendo o método:

```
processar(valor: number): void
```

Parte C – Implementações

Crie duas classes que implementam essa interface:

- PagamentoCartao
- PagamentoPix

Cada classe deve implementar o método processar.

Exemplo esperado:

```
class PagamentoPix implements MetodoPagamento {  
    processar(valor: number) {  
        console.log("Pagamento via PIX aprovado:", valor);  
    }  
}
```

Parte D – Criando a Factory

Crie uma classe chamada PagamentoFactory responsável por criar os objetos de pagamento.

Ela deve possuir o método:

```
criar(tipo: string): MetodoPagamento
```

Parte E – Utilizando a Factory

Utilize a fábrica para processar um pagamento.

Exemplo esperado:

```
const pagamento = PagamentoFactory.criar("pix");  
pagamento.processar(150);
```

Exercício 2 – Aplicando o Singleton Pattern

Contexto

Em aplicações backend, é comum existir uma classe responsável por representar a **conexão com o banco de dados**.

O problema é que, se o sistema criar várias instâncias dessa classe, podem ocorrer:

- desperdício de recursos
- múltiplas conexões desnecessárias
- dificuldade de controle da aplicação

Considere a implementação inicial a seguir:

```
class DatabaseConnection {  
    constructor() {
```



```
        console.log("Conectando ao banco de dados...");
    }

    query(sql: string): void {
        console.log("Executando SQL:", sql);
    }
}
```

Agora observe o uso dessa classe:

```
const db1 = new DatabaseConnection();
const db2 = new DatabaseConnection();

db1.query("SELECT * FROM usuarios");
db2.query("SELECT * FROM produtos");
```

Parte A – Identificando o problema

Responda:

- Qual problema pode ocorrer ao criar múltiplas instâncias da classe DatabaseConnection?
- Por que esse tipo de classe é um bom candidato ao uso do Singleton Pattern?

Parte B – Transformando a classe em Singleton

Modifique a classe DatabaseConnection para que:

- o construtor seja **privado**
- exista uma propriedade estática para armazenar a única instância
- exista um método estático getInstance() para retornar essa instância

Parte C – Testando o Singleton

Crie duas variáveis:

```
const db1 = DatabaseConnection.getInstance();
const db2 = DatabaseConnection.getInstance();
```

Depois:

- execute duas consultas com query()
- verifique se db1 e db2 apontam para o mesmo objeto

Exercício 3 – Aplicando o Abstract Factory Pattern

Contexto

Considere um sistema de e-commerce que pode ser utilizado em duas plataformas:

- Web
- Mobile

Cada plataforma possui **implementações específicas de métodos de pagamento**.

Plataforma	Cartão	Pix
Web	WebCartao	WebPix
Mobile	MobileCartao	MobilePix

O objetivo é criar um sistema capaz de gerar métodos de pagamento compatíveis com a plataforma utilizada. Para isso, será aplicado o Abstract Factory Pattern.

Parte A – Interface dos métodos de pagamento

Crie uma interface chamada `MetodoPagamento` contendo o método:

```
processar(valor: number): void
```

Parte B – Implementações para Web

Crie duas classes:

- `WebPagamentoCartao`
- `WebPagamentoPix`

Essas classes devem implementar a interface `MetodoPagamento`.

Parte C – Implementações para Mobile

Crie duas classes:

- `MobilePagamentoCartao`
- `MobilePagamentoPix`

Essas classes também devem implementar a interface `MetodoPagamento`.

Parte D – Interface da fábrica

Crie uma interface chamada `PagamentoFactory` contendo os métodos:

```
criarPagamentoCartao(): MetodoPagamento  
criarPagamentoPix(): MetodoPagamento
```

Parte E – Fábricas concretas

Implemente duas fábricas:

- `WebPagamentoFactory`
- `MobilePagamentoFactory`

Cada uma deve criar os objetos correspondentes à sua plataforma.

Parte F – Utilização da fábrica

Implemente uma função que utilize a fábrica para processar pagamentos.

Exemplo de uso esperado:

```
const factory = new WebPagamentoFactory();  
const pagamento = factory.criarPagamentoPix();  
pagamento.processar(200);
```

Exercício 4 – Aplicando o Builder Pattern

Contexto

Considere uma classe que representa uma requisição HTTP.

```
class HttpRequest {  
    constructor(  
        public url: string,  
        public method: string,  
        public headers: object,  
        public body: object,  
    ) {}  
}
```

Embora essa classe funcione, a criação de objetos pode se tornar difícil de ler e manter, principalmente quando há muitos parâmetros.

Exemplo:

```
const req = new HttpRequest(  
    "/usuarios",  
    "POST",  
    { Authorization: "Bearer token" },  
    { nome: "Ana", idade: 25 },  
);
```

À medida que o número de atributos cresce, esse tipo de construção se torna menos legível.

Para resolver esse problema, vamos aplicar o Builder Pattern.

Parte A – Identificando o problema

Responda:

- Por que construtores com muitos parâmetros podem dificultar a manutenção do código?
- Qual é a principal vantagem de construir o objeto em etapas?

Parte B – Criando a classe principal

Crie uma classe chamada `HttpRequest` com os atributos:

- url
- method
- headers
- body

Esses atributos podem ser definidos como opcionais.

Parte C – Criando o Builder

Crie uma classe chamada `HttpRequestBuilder` contendo:

- um atributo privado para armazenar o objeto em construção
- os métodos:
 - `setUrl(url: string)`

- `setMethod(method: string)`
- `setHeaders(headers: object)`
- `setBody(body: object)`
- um método `build()` que retorna o objeto final

Todos os métodos `set` devem retornar o próprio builder, permitindo **encadeamento de chamadas**.

Parte D – Construindo a requisição

Utilize o builder para montar a seguinte requisição:

- URL: `/usuarios`
- Método: `POST`
- Body: `{ nome: "Ana" }`

Exemplo esperado:

```
const request = new HttpRequestBuilder()
    .setUrl("/usuarios")
    .setMethod("POST")
    .setBody({ nome: "Ana" })
    .build();
```

Parte E – Exibindo o resultado

Imprima o objeto final no terminal.

Exercício 5 – Aplicando o Prototype Pattern

Contexto

Considere uma classe que representa um produto em um sistema de e-commerce.

```
class Produto {
    constructor(
        public nome: string,
        public preco: number,
        public categoria: string,
    ) {}
}
```

Suponha que o sistema precise criar vários produtos semelhantes, mudando apenas alguns atributos.

Exemplo:

- Notebook Dell
- Notebook Dell – promoção
- Notebook Dell – versão premium

Em vez de criar cada produto manualmente, podemos utilizar o Prototype Pattern para criar cópias de um objeto base.

Parte A – Identificando o problema

Responda:

- a) Por que pode ser útil copiar um objeto existente em vez de criar um novo objeto manualmente?
- b) Em quais situações a clonagem de objetos pode facilitar o desenvolvimento de um sistema?

Parte B – Implementando o Prototype

Modifique a classe Produto adicionando um método:

```
clone(): Produto
```

Esse método deve retornar uma cópia do objeto atual.

Parte C – Criando o protótipo

Crie um objeto base chamado produtoBase.

```
const produtoBase = new Produto("Notebook Dell", 3500, "Informática");
```

Parte D – Criando clones

Utilize o método clone() para criar dois novos produtos:

- um produto em promoção
- um produto premium

Exemplo esperado:

```
const produtoPromocao = produtoBase.clone();  
produtoPromocao.preco = 2999;  
  
const produtoPremium = produtoBase.clone();  
produtoPremium.preco = 4200;
```

Parte E – Exibindo os produtos

Imprima no console:

- produtoBase
- produtoPromocao
- produtoPremium